

Ringkasan Materi Pemrograman C++

Ringkasan Sintaks, Pola, dan Latihan

Disusun dari kumpulan materi PPK dan TLX Toki

Sudah disesuaikan dengan timeline pembelajaran

Isi dokumen

- Kerangka program, variabel, tipe data, operator, `cmath`
- Percabangan `if/else` dan `switch-case`
- Perulangan `for`, `while`, `do-while`, bersarang, `break/continue`
- Array satu dan dua dimensi, array paralel
- Fungsi: parameter *default*, *overloading*, ruang lingkup, rekursi, parameter array dan referensi
- `struct`, array of struct, string, dan karakter
- Pola algoritma: digit, Fibonacci, biner-desimal, pencarian, pola cetak

Referensi cepat untuk belajar dan mengerjakan soal

Daftar Isi

1	Ringkasan Dokumen	3
1.1	Kompilasi	3
2	Dasar Program C++	4
2.1	Kerangka Program	4
2.2	Output dan Input	4
2.3	Tipe Data dan Variabel	4
2.4	Operator Aritmatika dan <code>cmath</code>	5
2.5	Menyusun Ekspresi Rumus	5
3	Percabangan	7
3.1	Operator Relasional dan Logika	7
3.2	<code>if</code> , <code>else if</code> , <code>else</code>	7
3.3	<code>switch-case</code>	8
4	Perulangan	9
4.1	Operator Inkrement	9
4.2	<code>for</code>	9
4.3	<code>while</code> dan Pola Akumulator	10
4.4	Perulangan Bersarang dan Pola Cetak	11
4.4.1	Pola Grafis	12
4.5	<code>break</code> dan <code>continue</code>	12
4.6	Memilih <code>for</code> atau <code>while</code>	13
5	Array	14
5.1	Array Satu Dimensi	14
5.2	Array Paralel	15
5.3	Array Dua Dimensi	15
5.4	Pola Array yang Sering Muncul	16
6	Fungsi	17
6.1	Mengapa Menggunakan Fungsi?	17
6.2	Deklarasi dan Nilai Balik (<i>Return Value</i>)	18
6.3	Parameter <i>Default</i>	19
6.4	<i>Overloading</i> Function	19
6.5	Ruang Lingkup Variabel (<i>Scope</i>)	20
6.6	Rekursi	20
6.7	Parameter Array dan Referensi	21
7	Struct	22

7.1	Definisi dan Akses Anggota	22
7.2	Array of Struct	22
8	Pola Algoritma	24
8.1	Ekstraksi Digit: Modulus dan Pembagian	24
8.2	Deret Fibonacci	24
8.3	Konversi Biner $\leftrightarrow$ Desimal	25
9	Studi Kasus: Game Tebak Angka	27
10	Peta Latihan TLX TOKI	28
11	Glosarium Singkat	29

1 Ringkasan Dokumen

Dokumen ini berisi semua hal penting tentang C++ yang keluar di praktikum PPK (6 pertemuan terakhir) dan latihan-latihan di TLX TOKI. Tiap topik sudah disediakan contoh kode agar langsung bisa dicoba sendiri.

1.1 Kompilasi

Kalau pakai Windows, compile programnya seperti gini:

```
# compile satu file cpp
g++ nama_file.cpp

# secara otomatis akan menghasilkan a.exe
# jalankan
./a.exe

# kompilasi dengan hasil eksekusi nama file yang berbeda sekaligus menjalankannya
g++ nama_file.cpp -o kode_pertamaku.exe
./kode_pertamaku.exe
```

Kalau pakai MacOS atau Linux, prosesnya sama saja:

```
# compile satu file cpp
g++ nama_file.cpp

# secara otomatis akan menghasilkan a.out
# jalankan
./a.out

# kompilasi dengan hasil eksekusi nama file yang berbeda sekaligus menjalankannya
g++ nama_file.cpp -o kode_pertamaku.out
./kode_pertamaku.out
```

2 Dasar Program C++

2.1 Kerangka Program

```

1 #include <iostream>
2 using namespace std;      // boleh dihilangkan, tulis std::cout
3
4 int main()
5 {
6     cout << "Halo, dunia!" << endl;
7     return 0;              // opsional di dalam main()
8 }

```

Listing 1: Kerangka program minimum

Hasil:

Halo, dunia!

- `#include <iostream>` — ini memberi akses ke `cin` (buat input) dan `cout` (buat output).
- `int main()` — ini titik awal program jalan; hanya boleh ada satu.
- `cout << ...;` — untuk menghasilkan output, `cin >> ...;` — untuk menerima input dari keyboard.
- `endl` untuk pindah baris baru; `"\n"` juga sama, tapi lebih ringan secara komputasi.

2.2 Output dan Input

```

1 cout << "Panjang: ";
2 cin >> panjang;
3 cout << "\nLebar: ";
4 cin >> lebar;
5
6 cout << "\nLuas = " << panjang * lebar << endl;

```

Listing 2: Menyusun keluaran dan membaca masukan

- Operator `<<` bisa ditumpuk: output disusun dari kiri ke kanan.
- Input ngikutin tipe variabel. `cin >>` ke `int` dengan input 3.14 hanya baca 3 aja.
- `cin >>` ke `bool` hanya nerima 0 atau 1.
- Bilangan desimal harus dibaca ke `double`, bukan `int`.

2.3 Tipe Data dan Variabel

Tipe	Contoh	Kisaran/ukuran	Kegunaan
<code>int</code>	3, -17	$\pm 2,1 \times 10^9$ (32 bit)	bilangan bulat
<code>long long</code>	2083164274682234208	$\pm 9,2 \times 10^{18}$ (64 bit)	bilangan besar
<code>double</code>	4.6, 3.14	≈ 15 digit signifikan	bilangan berkoma
<code>char</code>	'A'	1 byte	satu karakter
<code>string</code>	"Halo"	bervariasi	teks
<code>bool</code>	true/false	1 byte	nilai logika

```

1 int    a = 3;
2 double b = 4.6;
3 char   c = 'A';
4 string d = "Halo";
5 bool   e = true;
6
7 const double PI = 3.14;
8 PI = 3.14159;           // ERROR: konstanta tidak boleh di-assign ulang

```

Listing 3: Deklarasi, inisialisasi, dan konstanta

Jebakan

Karakter pakai kutip tunggal ('A'), string pakai kutip ganda ("A"). 'AB' bukan char yang valid. Selain itu, jangan lupa sertakan `#include <string>` saat pakai `string`.

2.4 Operator Aritmatika dan `cmath`

Operator	Arti	Contoh
+ - * /	empat operasi dasar	10 / 3 → 3
%	sisanya bagi (modulus)	10 % 3 → 1

Fungsi <code>cmath</code>	Kegunaan
<code>pow(a,b)</code> , <code>sqrt(a)</code>	pangkat, akar
<code>abs(x)</code>	nilai mutlak
<code>ceil(x)</code> , <code>floor(x)</code>	pembulatan ke atas/bawah
<code>round(x)</code>	pembulatan terdekat
<code>fmax(a,b)</code> , <code>fmin(a,b)</code>	maksimum/minimum
<code>M_PI</code>	nilai π

Jebakan

Pembagian bulat. `7 / 2` hasilnya 3, bukan 3.5. Agar mendapatkan hasil desimal, salah satu operand harus `double`: `7.0 / 2`. Contoh lain: rumus suhu $(f - 32) * 5 / 9$ hanya benar kalau `f` bertipe `double`.

Jebakan

`sqrt(a)` hanya terima *satu* argumen. Menulis dengan cara `sqrt(a, b)` membuat program gagal di-*compile*. Begitu juga `22 / 7` tetap 3 — lebih baik pakai `M_PI`.

2.5 Menyusun Ekspresi Rumus

```

1 double base, height, area;
2
3 cout << "INPUT base: "; // anggap saja ini x
4 cin >> base;
5 cout << "\nINPUT height: "; // anggap saja ini y
6 cin >> height;
7
8 area = 0.5 * base * height;

```

```
9 cout << "\nTriangle area = " << area;
```

Listing 4: Pola dasar: deklarasi -> baca -> hitung -> cetak

Hasil:

INPUT base: x # dari input user

INPUT height: y # dari input user

Triangle area = ... # hasil dari $0.5 * x * y$

Pola Umum

Hampir semua soal awal polanya sama: **deklarasi variabel** → **baca input** → **hitung rumus** → **cetak hasil**. Bedakan tipe data dari awal: kurs mata uang pakai `int`, luas segitiga pakai `double`.

3 Percabangan

3.1 Operator Relasional dan Logika

Relasional	Logika	Nama	Benar bila ...
< > <= >=	&&	AND	kedua operand benar
== !=		OR	setidaknya satu benar
— hasil selalu bool	!	NOT	membalik nilai

```

1 cout << (5 < 3) << endl; // 0 (salah)
2 cout << (5 > 3) << endl; // 1 (benar)
3 cout << (5 == 3) << endl; // 0

```

Listing 5: Ekspresi relasional dicetak sebagai 1/0

Hasil:

```

0
1
0

```

Jebakan

`==` Untuk membandingkan value, `=` untuk memberikan/assign value. Kalau kalian menuliskan `if (password = saved_password)`, hasilnya selalu bernilai `true` sekaligus merusak/merubah value `password`.

3.2 if, else if, else

```

1 if (time < 12) {
2     cout << "Good morning!";
3 }
4 else if (time < 18) {
5     cout << "Good afternoon!";
6 }
7 else {
8     cout << "Good evening!";
9 }

```

Listing 6: Rantai percabangan

- Kondisi dicek berurutan; *hanya satu* cabang yang dijalankan.
- Urutan menentukan hasil: `time = 9` berhenti di kondisi pertama, `time = 15` jatuh ke kondisi kedua.
- Percabangan boleh dibuat bersarang/*nested* untuk validasi, lalu tulis penerapan logika di dalamnya.

```
1 if (points > 0 && points < 100) {
2     if (points <= 20)      cout << "Score = E";
3     else if (points <= 40) cout << "Score = D";
4     else if (points <= 60) cout << "Score = C";
5     else if (points <= 80) cout << "Score = B";
6     else                  cout << "Score = A";
7 }
8 else {
9     cout << "Points out of scope";
10 }
```

Listing 7: Validasi rentang lalu penetapan nilai huruf

Jebakan

Pada kode di atas 0 dan 100 sendiri ikut ditolak. Bila rentang termasuk kedua ujungnya, tulis `points >= 0 && points <= 100`. Selalu uji *edge cases* saat menulis validasi.

3.3 switch-case

```
1 switch (memberId) {
2     case 1:
3         memberName = "Ani";
4         break;
5     case 2:
6         memberName = "Budi";
7         break;
8     case 3:
9         memberName = "Wati";
10        break;
11    default:
12        cout << "Member is not recognized!" << endl;
13 }
```

Listing 8: switch dengan break dan default

- Ekspresi harus bertipe `int`, `char`, atau enumerasi (pilih salah satu tipe data yang sama untuk satu program).
- Tanpa `break`, eksekusi *jatuh* ke `case` berikutnya (*fall-through*).
- `default` bersifat opsional tetapi sangat disarankan agar semua kemungkinan tertutup.

```
1 switch (dayNum) {
2     case 1: cout << "Monday";    break;
3     case 2: cout << "Tuesday";   break;
4     // ... kasus 3--6 ...
5     case 7: cout << "Sunday";    break;
6     default: cout << "Wrong number!";
7 }
```

Listing 9: Aktivitas: nama hari dari bilangan 1–7

Tips

Gunakan `if-else` untuk kondisi rentang, misalnya `(x > 10 && x <= 20)` dan `switch` untuk pencocokan banyak nilai tetap.

4 Perulangan

4.1 Operator Inkrement

Ekspresi	Setelah dieksekusi	Catatan
<code>x++</code>	<code>x = x + 1</code>	<i>post</i> : nilai lama dipakai dulu
<code>++x</code>	<code>x = x + 1</code>	<i>pre</i> : bertambah dulu, baru dipakai
<code>x += n</code>	<code>x = x + n</code>	penambahan bertahap (akumulator)
<code>x--</code>	<code>x = x - 1</code>	

```

1 int x = 5;
2 cout << x << endl; // 5
3 cout << x++ << endl; // 5 (nilai lama dicetak dulu)
4 cout << x << endl; // 6

```

Listing 10: Perbedaan perilaku `x++`

Hasil:

```

5
5
6

```

4.2 for

```

1 for (int i = 1; i <= 10; i++) { // inisialisasi; kondisi; inkrement
2     cout << i << endl;
3 }

```

Listing 11: Perulangan Naik

Hasil:

```

1
2
3
4
5
6
7
8
9
10

```

```
1 for (int i = 10; i >= 3; i--) { // urutan terbalik, dengan pembatasan akhir adalah 3
2     cout << i << endl;
3 }
```

Listing 12: Perulangan Turun

Hasil:

```
10
9
8
7
6
5
4
3
```

- Inisialisasi dijalankan sekali di awal, kondisi diperiksa tiap awal iterasi, inkrement dijalankan di akhir tiap iterasi.
- Pindahkan batas ke konstanta (`const int BATAS = 10;`) agar kode mudah diubah.

4.3 while dan Pola Akumulator

```
1 int i = 0;
2 while (i <= 10) {
3     cout << i << "\t";
4     i++; // tanpa baris ini -> infinite loop
5 }
```

Listing 13: while dengan penghitung

Hasil:

```
0 1 2 3 4 5 6 7 8 9 10
```

```
1 int number, input, sum = 0, i = 1;
2
3 cout << "Input how many numbers = ";
4 cin >> number;
5
6 while (i <= number) {
7     cout << "Input number " << i << " = ";
8     cin >> input;
9     sum += input;
10    i++;
11 }
12 cout << "Sum = " << sum << endl;
```

Listing 14: Akumulasi N bilangan dari pengguna

Hasil:

```
Input how many numbers = Sum = 0
```

Pola Umum

Tiga pola `while` yang paling sering muncul:

1. **Penghitung (*counter*)** — berhenti setelah N iterasi, misalnya `while (baris < 3)`.
2. **Akumulator** — menjumlahkan nilai tiap iterasi, awali `total = 0` atau nilai awal yang benar (`total = luas_awal`).
3. **Sentinel/kondisi nilai** — berhenti saat syarat tercapai: `while (total < 800)`, `while (betina <= jantan * 10)`.

```
1 bool repeat = true;
2 while (repeat) {
3     cout << "Enter a number: ";
4     cin >> number;
5     cout << "Want to repeat? ";
6     cin >> repeat;           // 0 = berhenti, 1 = ulang
7 }
```

Listing 15: Loop yang dikendalikan pengguna (flag)

```
1 do {
2     // isi permainan
3 } while (inginBermainLagi);
```

Listing 16: do-while: minimal sekali jalan

4.4 Perulangan Bersarang dan Pola Cetak

```
1 for (int i = 1; i <= 5; i++) {           // luar: 5 baris
2     for (int j = 1; j <= 7; j++) {       // dalam: 7 kolom per baris
3         cout << j << "\t";
4     }
5     cout << endl;
6 }
```

Listing 17: Perulangan bersarang: tabel angka

Hasil:

```
1 2 3 4 5 6 7
1 2 3 4 5 6 7
1 2 3 4 5 6 7
1 2 3 4 5 6 7
1 2 3 4 5 6 7
```

```

1 for (int i = 1; i <= 5; i++) {
2     for (int j = i; j <= 7; j++) {        // j mulai dari i
3         cout << j << "\t";
4     }
5     cout << endl;
6 }

```

Listing 18: Inner loop memakai nilai outer loop

Hasil:

```

1 2 3 4 5 6 7
2 3 4 5 6 7
3 4 5 6 7
4 5 6 7
5 6 7

```

Pola Grafis

```

1 // segitiga: i buah bintang, sisanya titik
2 for (int i = 1; i <= N; i++) {
3     for (int j = 1; j <= N; j++)
4         cout << (j <= i ? "*" : ".");
5     cout << endl;
6 }
7
8 // bingkai: bintang hanya pada tepi
9 for (int i = 1; i <= N; i++)
10    for (int j = 1; j <= N; j++)
11        cout << (i == 1 || i == N || j == 1 || j == N ? "*" : ".");
12
13 // silang: diagonal utama atau diagonal antisamping
14 cout << (i == j || i + j == N + 1 ? "*" : ".");

```

Listing 19: Segitiga, bingkai, dan silang

Jebakan

Syarat diagonal di atas memakai basis-1 (indeks 1..N). Jika loop diubah ke basis-0, syarat kanan-atas menjadi $i + j == N - 1$.

4.5 break dan continue

break	continue
Keluar dari perulangan sepenuhnya	Melompati sisa iterasi ini, lanjut ke iterasi berikutnya
Cocok saat solusi sudah ketemu	Cocok saat satu kandidat pasti tidak valid

```

1 for (int luas = 100000; luas <= 200000; luas++) {
2     if (luas % 2 != 0) continue;        // bukan genap: lewati
3
4     bool kuadrat_sempurna = false;
5     for (int k = 1; k <= luas; k++) {
6         if (k * k == luas) {

```

```

7         kuadrat_sempurna = true;
8         break;                                // sudah ketemu: berhenti
9     }
10 }
11 if (kuadrat_sempurna) { cout << luas; break; }
12 }

```

Listing 20: continue menyaring kandidat tak valid

Hasil:

101124

```

1 if (tebakan < 1 || tebakkan > 100) {
2     cout << "Input tidak valid!" << endl;
3     continue;                                // tidak dihitung sebagai percobaan
4 }
5 riwayat[percobaan] = tebakkan;
6 percobaan++;

```

Listing 21: continue menolak input pengguna

4.6 Memilih for atau while

for	while
Jumlah iterasi diketahui sejak awal	Jumlah iterasi bergantung kondisi
Inisialisasi, kondisi, inkrement dalam satu baris	Ditulis manual, bebas bentuk
Contoh: cetak 1–100, telusuri array	Contoh: menebak sampai benar, akumulasi sampai batas tercapai

5 Array

5.1 Array Satu Dimensi

```
1 string member[3] = { "Ani", "Budi", "Wati" };
2 cout << member[2] << endl;           // Wati   (indeks diawali 0)
3
4 // cara 1: indeks
5 for (int i = 0; i < 3; i++) {
6     cout << member[i] << endl;
7 }
8
9 // cara 2: range-based for (C++11 ke atas)
10 for (string m : member) {
11     cout << m << endl;
12 }
```

Listing 22: Deklarasi, inisialisasi, dan akses indeks

Hasil:

Hasil dari memanggil value berdasarkan pemilihan indeks

Wati

Hasil dari cara 1: looping index

Ani

Budi

Wati

Hasil dari cara 2: range-based for

Ani

Budi

Wati

```
1 int numbers[5] = {10, 20, 30, 40, 50};
2 int sum = 0;
3
4 for (int i = 0; i < 5; i++) {
5     sum += numbers[i];
6 }
7 cout << "sum = " << sum << endl;     // 150
```

Listing 23: Akumulasi elemen array

Hasil:

sum = 150

Jebakan

Sesuaikan penggunaan indeks sesuai studi kasus yang disediakan pada soal-soal. Pada umumnya, indeks sah hanya 0 sampai $n-1$. Loop harus memakai $i < n$, bukan $i \leq n$.

5.2 Array Paralel

Beberapa array berukuran sama dapat mewakili “satu daftar objek”: indeks yang sama merujuk pada objek yang sama.

```

1 string title[5] = {"Harry Potter", "Algorithm", "Calculus",
2                   "Sherlock Holmes", "Supernova"};
3 int price[5]    = {250000, 85000, 130000, 270000, 180000};
4 bool available[5] = {true, true, false, true, true};
5
6 for (int i = 0; i < 5; i++) {
7     if (!available[i]) continue;           // buku tidak tersedia: lewati
8     cout << title[i] << " - " << price[i] << endl;
9 }

```

Listing 24: Toko buku berbasis array paralel

Hasil:

```

Harry Potter - 250000
Algorithm - 85000
Sherlock Holmes - 270000
Supernova - 180000

```

5.3 Array Dua Dimensi

```

1 string letters[2][4] = {
2     { "A", "B", "C", "D" },
3     { "E", "F", "G", "H" }
4 };
5
6 for (int i = 0; i < 4; i++) {           // i: kolom
7     for (int j = 0; j < 2; j++) {       // j: baris
8         cout << letters[j][i] << "\t";
9     }
10    cout << "\n";
11 }

```

Listing 25: Deklarasi dan penelusuran array 2D

Hasil:

```

A E
B F
C G
D H

```

```

1 for (int i = 0; i < 5; i++) {
2     cout << i << "\t";
3     for (int j = 0; j < 6; j++) {
4         if (available[i][j]) {
5             cout << "-" << "\t";
6         }
7         else {
8             cout << "V" << "\t";
9         }
10    }
11    cout << endl;

```

```

12 }
13
14 cout << endl;
15
16 cout << "Select row number: ";
17 cin >> row;
18 cout << "Select column number: ";
19 cin >> column;
20
21 if (!available[row][column]) {
22     cout << "Sorry, the seat is occupied" << endl;
23 }
24 else {
25     available[row][column] = false;    // tandai kursi sudah dipesan
26     cout << "You have purchased seat [" << row << "," << column << "]" << endl;
27 }

```

Listing 26: Array 2D bool sebagai peta status *seatmap*

Jebakan

Kode *seatmap* tidak mengecek apakah *row* dan *column* berada dalam rentang 0..4 dan 0..5. Akses di luar array = *undefined behavior*. Selalu validasi indeks yang datang dari pengguna.

5.4 Pola Array yang Sering Muncul

```

1 // 1) nilai terbesar: linear scan
2 int largest = luas[0];
3 for (int i = 1; i < 5; i++)
4     if (luas[i] > largest) largest = luas[i];
5
6 // 2) pasangan bersebelahan yang sama (batas N-1!)
7 for (int i = 0; i < 9; i++)
8     if (data[i] == data[i + 1]) hasil++;
9
10 // 3) total kumulatif (prefix sum) --- cetak DI DALAM loop
11 for (int i = 0; i < 10; i++) {
12     total += beli[i] - jual[i];
13     cout << total << endl;
14 }

```

Listing 27: Nilai maksimum, tetangga sama, dan total kumulatif

```

1 int harga_jual[3] = {100, 120, 150};
2 int total[3] = {0, 0, 0};
3
4 for (int i = 0; i < 4; i++)
5     for (int j = 0; j < 3; j++)
6         total[j] += luas[i][j];
7
8 for (int j = 0; j < 3; j++)
9     cout << total[j] * harga_jual[j] << endl;

```

Listing 28: Agregasi per kolom matriks dikalikan harga

6 Fungsi

6.1 Mengapa Menggunakan Fungsi?

Dalam pemrograman, fungsi digunakan untuk **mengelompokkan suatu proses yang memiliki tujuan tertentu** sehingga proses tersebut dapat dipanggil kembali ketika dibutuhkan. Dengan demikian, kita tidak perlu menuliskan ulang logika yang sama di banyak tempat.

Sebagai contoh, ada aplikasi perhitungan tarif bebek, kita perlu menentukan harga berdasarkan jumlah bebek. Logika tersebut dapat ditulis langsung di dalam program:

```
1 if (total_jantan < 10)
2     biaya_jantan = 100000 * total_jantan;
3 else if (total_jantan <= 50)
4     biaya_jantan = 75000 * total_jantan;
5 else
6     biaya_jantan = 50000 * total_jantan;
7
8 if (total_betina < 10)
9     biaya_betina = 100000 * total_betina;
10 else if (total_betina <= 50)
11     biaya_betina = 75000 * total_betina;
12 else
13     biaya_betina = 50000 * total_betina;
```

Listing 29: Logika tarif ditulis secara langsung

Perhatikan bahwa kedua bagian program tersebut melakukan **proses yang sama (repetitif)**. Perbedaannya hanya pada data yang digunakan, yaitu jumlah bebek jantan dan jumlah bebek betina. Jika tarif berubah, misalnya harga untuk jumlah bebek di bawah 10 ekor berubah dari Rp100.000 menjadi Rp120.000, maka kita harus mengubah kode di beberapa tempat.

Masalah tersebut dapat diatasi dengan menerapkan sebuah fungsi:

```
1 int biaya(int total_bebek) {
2     int harga_bebek;
3
4     if (total_bebek < 10)
5         harga_bebek = 100000;
6     else if (total_bebek <= 50)
7         harga_bebek = 75000;
8     else
9         harga_bebek = 50000;
10
11     return harga_bebek * total_bebek;
12 }
```

Listing 30: Logika tarif diekstrak menjadi fungsi

Sekarang, fungsi `biaya()` bertugas **menghitung biaya berdasarkan jumlah bebek**. Fungsi tersebut dapat digunakan berkali-kali dengan data yang berbeda:

```
1 int biaya_jantan = biaya(total_jantan);
2 int biaya_betina = biaya(total_betina);
```

Listing 31: Menggunakan fungsi untuk data yang berbeda

Dengan pendekatan ini, logika tarif hanya ditulis **satu kali**. Jika aturan tarif berubah, kita cukup memperbaiki fungsi `biaya()`. Bagian program lain yang menggunakan fungsi tersebut tidak perlu diubah.

Tips

Inti penggunaan fungsi adalah memisahkan “apa yang dilakukan” dari “data apa yang digunakan”.

Fungsi `biaya()` mengetahui *bagaimana* cara menghitung biaya, sedangkan program utama hanya perlu memberikan *data* berupa `total_bebek`.

Mengapa fungsi membuat program lebih baik? Penggunaan fungsi memberikan beberapa keuntungan penting:

1. **Menghindari pengulangan kode (DRY).**

Logika yang sama cukup ditulis satu kali dan dapat digunakan berkali-kali. Prinsip ini sering disebut *Don't Repeat Yourself* (DRY).

2. **Memudahkan perbaikan.**

Jika terdapat kesalahan atau perubahan aturan, kita cukup memperbaiki fungsi yang berkaitan. Kita tidak perlu mencari dan mengubah setiap salinan kode yang sama.

3. **Membuat program lebih mudah dibaca.**

Dibandingkan melihat banyak baris kode perhitungan, pembaca cukup melihat `biaya(total_bebek)` untuk memahami bahwa program sedang menghitung biaya.

4. **Memungkinkan penggunaan kembali kode.**

Fungsi yang sudah dibuat dapat dipanggil dari banyak bagian program tanpa perlu menyalin implementasinya.

5. **Membagi program menjadi bagian-bagian kecil.**

Program yang besar dapat dipecah menjadi beberapa fungsi yang masing-masing memiliki tanggung jawab tertentu. Hal ini membuat program lebih mudah dikembangkan dan diuji.

Tips

Fungsi sebaiknya memiliki **satu tugas yang jelas**. Misalnya, `biaya()` bertugas menghitung biaya, bukan sekaligus membaca input, mencetak hasil, dan menghitung denda.

Prinsip ini akan semakin penting ketika program menjadi lebih besar. Kita dapat membuat fungsi yang saling bekerja sama, misalnya: `total_denda()` memanggil `denda_pada_hari()`, yang kemudian menggunakan `selisih()` untuk menghitung perbedaan jumlah hari.

Dengan demikian, fungsi bukan sekadar cara untuk “memendekkan kode”. Fungsi merupakan cara untuk **mengorganisasi program berdasarkan tanggung jawabnya**. Semakin besar program yang dibuat, semakin penting kemampuan untuk memecah masalah besar menjadi fungsi-fungsi kecil yang jelas dan dapat digunakan kembali.

6.2 Deklarasi dan Nilai Balik (*Return Value*)

```
1 void sayHello() {  
2     cout << "Hello, good morning!" << endl;  
3 }  
4  
5 int rectangleArea(int width, int height) {  
6     return width * height;  
7 }
```

```
8
9 int main() {
10     sayHello();
11     cout << rectangleArea(5, 4);
12 }
```

Listing 32: Fungsi void dan fungsi bernilai balik

Hasil:

```
Hello, good morning!
20
```

6.3 Parameter *Default*

```
1 void sayHello(string name = "Fulan", int number = 7) {
2     cout << "Hello, good morning " << name << endl;
3     cout << "You have selected number " << number << endl;
4 }
5
6 int main() {
7     sayHello();                // memakai kedua nilai default
8     sayHello("Ani", 5);        // nilai default tertimpa
9 }
```

Listing 33: Parameter bertanda nilai *default*

Hasil:

```
Hello, good morning Fulan
You have selected number 7
Hello, good morning Ani
You have selected number 5
```

- Parameter bertanda *default* harus berada di urutan paling belakang.
- Nilai *default* harus berupa nilai konstan yang diketahui saat kompilasi.

6.4 *Overloading* Function

```
1 int sumNumbers(int num1, int num2) {
2     return num1 + num2;
3 }
4
5 int sumNumbers(int num1, int num2, int num3) {
6     return num1 + num2 + num3;
7 }
8
9 int main() {
10     cout << sumNumbers(4, 5) << endl;    // 9
11     cout << sumNumbers(4, 5, 6) << endl; // 15
12 }
```

Listing 34: Dua fungsi bernama sama dengan parameter berbeda

Hasil:

9
15

- Yang membedakan adalah jumlah atau tipe parameter, bukan nilai balik.
- Contoh lain: `printDate(int,int,int)` vs `printDate(int,string,int)`.

6.5 Ruang Lingkup Variabel (*Scope*)

```

1 int x = 5;                                // global
2
3 void printX() {
4     int x = 7;                            // lokal: menutupi global
5     cout << "X inside function = " << x << endl;
6 }
7
8 int main() {
9     cout << "X global = " << x << endl;    // 5
10    printX();                             // 7
11 }
```

Listing 35: Variabel global ditutupi variabel lokal (*shadowing*)

Hasil:

X global = 5
X inside function = 7

Tips

Variabel lokal hanya hidup di dalam blok tempat ia dideklarasikan dan dibebaskan begitu fungsi selesai. Variabel global terbaca oleh semua fungsi — nyaman untuk data yang jarang berubah (mis. `bebek[11]`), tetapi membuat kode sulit dilacak bila dimodifikasi banyak fungsi.

6.6 Rekursi

```

1 int sum(int number) {
2     if (number > 0) {
3         return number + sum(number - 1);
4     } else {
5         return 0;                            // base case: fungsi berhenti
6     }
7 }
8
9 int fact(int num) {
10    if (num > 1) return num * fact(num - 1);
11    return 1;
12 }
```

Listing 36: Rekursi dengan *base case*

Jebakan

Rekursi harus memiliki kondisi berhenti (*base case*). Tanpa kondisi ini, fungsi akan terus memanggil dirinya sendiri sampai memori penuh dan menyebabkan *stack overflow*. Perhitungan faktorial juga bisa dilakukan tanpa rekursi, yaitu dengan menggunakan perulangan.

6.7 Parameter Array dan Referensi

```
1 void tampilkanRiwayat(int riwayat[], int jumlahPercobaan) {  
2     for (int i = 0; i < jumlahPercobaan; i++) {  
3         cout << "Percobaan " << i + 1 << ": " << riwayat[i] << endl;  
4     }  
5 }
```

Listing 37: Fungsi yang menerima array

```
1 void tukar(int &a, int &b) {  
2     int temp = a;  
3     a = b;  
4     b = temp;  
5 }
```

Listing 38: Menukar dua nilai dengan parameter referensi

Jebakan

Tanda & membuat **a** dan **b** merujuk langsung ke variabel asli. Jadi, ketika nilainya ditukar di dalam fungsi, nilai variabel aslinya juga ikut berubah. Tanpa &, yang ditukar hanya salinannya sehingga nilai asli tetap sama.

7 Struct

7.1 Definisi dan Akses Anggota

```
1 struct Member {
2     int memberID;
3     string name;
4 };
5
6 struct Book {
7     int bookID;
8     string title;
9     int price;
10 };
11
12 Member member1 = {1, "Ani"};
13 Book book1 = {1, "Harry Potter", 250000};
14
15 cout << member1.name << " membeli " << book1.title;
```

Listing 39: Definisi struct dan inisialisasi aggregate

Hasil:

Ani membeli Harry Potter

- `struct` adalah *kartu data*: beberapa field dengan tipe yang mungkin berbeda, disatukan dalam satu nama.
- Anggota diakses dengan tanda titik: `member1.memberID`.
- Tanpa `public`, `struct` bersifat `public` secara bawaan (berbeda dengan `class`).

7.2 Array of Struct

```
1 struct student {
2     int niu;
3     string fullName;
4     int year;
5     char gender;
6 };
7
8 student mhsTRPL[3];
9 mhsTRPL[0].niu = 555151;
10 mhsTRPL[0].fullName = "Maul";
11 mhsTRPL[0].year = 2026;
```

Listing 40: Menyimpan banyak data bertipe sama

```
1 for (int i = 0; i < total; i++) {
2     cout << i + 1 << "\t" << mhs[i].niu << "\t"
3         << mhs[i].fullName << "\t" << mhs[i].year << "\t"
4         << mhs[i].gender << endl;
5 }
```

Listing 41: Menampilkan array of struct sebagai tabel

Jebakan

Pada potongan kode tersebut deklarasinya `student mhsTRPL[total];` — sebuah *variable-length array* yang bukan bagian standar C++ — lalu loop ditulis `for (int i = 1; i <= total; i++)`, sehingga `mhsTRPL[total]` berada di luar batas array. Tulis loop `for (int i = 0; i < total; i++)`.

8 Pola Algoritma

8.1 Ekstraksi Digit: Modulus dan Pembagian

```

1 int sisa = num % 100;      // 341 -> 41   (potong dari kanan)
2 int digit_tengah = sisa / 10; // 41 -> 4   (potong dari kiri)

```

Listing 42: Mengambil digit tengah bilangan tiga digit

Tips

Pola “*potong dari kanan dengan %, potong dari kiri dengan /*” dipakai berulang: mengambil digit, memeriksa habis dibagi, dan konversi basis.

8.2 Deret Fibonacci

```

1 nextNum = num1 + num2;
2 num1 = num2;
3 num2 = nextNum;
4
5 cout << nextNum;
6 if (i < number) cout << ", ";

```

Listing 43: Fibonacci iteratif

- Dua bilangan pertama ditangani khusus (dicetak langsung), iterasi dimulai dari bilangan ketiga.
- Kuncinya adalah pertukaran tiga variabel `num1`, `num2`, `nextNum` — urutan pertukaran menentukan hasil.

Contoh kode lengkap untuk menghitung deret Fibonacci:

```

1  /* =====
2     Konversi Bilangan Biner dan Desimal
3  ===== */
4  #include <iostream>
5  using namespace std;
6
7  int main() {
8      /* =====
9         Inisiasi angka yang akan diinput, variabel
10         num1, num2, dan nextNum, karena dalam hukum Fibonacci
11         baris selanjutnya adalah hasil dari penjumlahan dari
12         2 angka sebelumnya
13         ===== */
14     int number, num1 = 0, num2 = 1, nextNum = 0;
15     cout << "Input number: ";
16     cin >> number;
17
18     /* =====
19         Pengantar output Fibonacci Sequence
20         ===== */
21     cout << "Fibonacci sequence:" << endl;
22
23     /* =====
24         Looping dijalankan sampai nilai batasnya (nilai index) sama dengan input number
25         ===== */
26     for (int i = 1; i <= number; i++) {

```

```

27      /* =====
28         Mencetak 2 angka pertama, yaitu 0 dan 1
29         ===== */
30      if (i == 1) {
31          cout << num1 << ", ";
32          continue;
33      }
34      else if (i == 2) {
35          cout << num2 << ", ";
36          continue;
37      }
38
39      /* =====
40         Angka selanjutnya (variabel nextNum) adalah hasil
41         dari penjumlahan dua angka pertama yang sudah dicetak
42         ===== */
43      nextNum = num1 + num2;
44      num1 = num2;
45      num2 = nextNum;
46
47      /* =====
48         Cetak angka selanjutnya
49         ===== */
50      cout << nextNum;
51
52      if (i < number) {
53          cout << ", ";
54      }
55  }
56
57  cout << "\n";
58
59  return 0;
60 }

```

Listing 44: Kode Lengkap untuk Mencetak Deret Fibonacci

8.3 Konversi Biner ↔ Desimal

```

1  int binaryToDecimal(string biner) {
2      int decimal = 0;
3      for (char digit : biner) {
4          decimal = decimal * 2 + (digit - '0');
5      }
6      return decimal;
7  }

```

Listing 45: Biner ke desimal

```

1  string hasil = "";
2  while (desimal > 0) {
3      hasil = char((desimal % 2) + '0') + hasil;    // dimasukkan di depan
4      desimal = desimal / 2;
5  }

```

Listing 46: Desimal ke biner (pembagian bertahap)

- `digit - '0'` mengubah karakter digit menjadi nilai numeriknya: `'1' - '0' → 1`.
- Sebaliknya, `nilai + '0'` mengubah angka menjadi karakter.
- Validasi tiap karakter harus `'0'` atau `'1'` sebelum diproses.

Kode lengkap untuk program konversi biner-desimal

```
1 long long panjang = 2002419288;  
2 long long tinggi = 1040323716;  
3  
4 long long luas = panjang * tinggi; // 2083164274682234208  
5 cout << luas << endl;
```

Listing 47: long long mencegah overflow

Hasil:

2083164274682234208

```
1 double alas = 1213;  
2 double tinggi = 1619;  
3  
4 double luas = alas * tinggi / 2;  
5 cout << fixed << setprecision(1) << luas << endl; // 981923.5
```

Listing 48: Bilangan desimal dengan presisi tetap

Jebakan

- Hasil perkalian dua bilangan besar harus `long long`; jika satu faktor tetap `int`, operasi dihitung 32 bit dan melampaui 2 147 483 647 (hasil salah).
- Tanpa `fixed`, `setprecision(1)` berarti *satu digit signifikan*, bukan satu angka di belakang koma — keluaran bisa berubah menjadi notasi ilmiah.
- Pembagian dua `int` tetap `int`: `1963847 / 2` bernilai 981923.

9 Studi Kasus: Game Tebak Angka

Pada pertemuan praktikum terakhir, kita membuat sebuah *mini project* yang merangkum hampir seluruh materi: fungsi, array sebagai parameter, konstanta, ruang lingkup, perulangan, serta break/continue. Dokumen ini menggunakan **Game Tebak Angka** sebagai contoh dari penerapan materi yang sudah didapatkan sebelumnya.

```
1 const int MAKSIMAL_PERCOBAAN = 5;
2
3 void mulaiGame() {
4     int angkaRahasia = rand() % 100 + 1;    // 1--100
5     int riwayat[MAKSIMAL_PERCOBAAN];
6     int percobaan = 0;
7     bool menang = false;
8
9     while (percobaan < MAKSIMAL_PERCOBAAN && !menang) {
10         cout << "Tebakan: ";
11         cin >> tebak;
12
13         if (tebak < 1 || tebak > 100) {      // validasi
14             cout << "Input tidak valid!" << endl;
15             continue;                      // tidak dihitung
16         }
17
18         riwayat[percobaan] = tebak;         // simpan riwayat
19         percobaan++;
20
21         if (tebak == angkaRahasia) {
22             menang = true;
23             break;                          // langsung berhenti
24         }
25         else if (tebak < angkaRahasia)
26             cout << "Petunjuk: Tebakan terlalu kecil!" << endl;
27         else
28             cout << "Petunjuk: Tebakan terlalu besar!" << endl;
29     }
30 }
```

Listing 49: Inti permainan: validasi, penyimpanan, dan percobaan

Untuk *source code* dari file diatas ada pada link GitHub berikut:

<https://github.com/genta-bahana-nagari/mini-project-task-cpp>

Tips

Potongan kode ini memakai `srand(time(0)) + rand()`, gaya lama yang telah digantikan `<random>` pada C++ modern. Untuk latihan dasar, cara tersebut masih berfungsi.

Notes

Karena ini menggunakan referensi proyek kelompok, bisa disesuaikan materinya dengan proyek milik kelompok anda.

10 Peta Latihan TLX TOKI

TLX Toki sangat membantu untuk memahami keseluruhan materi C++ beserta implementasinya. Dibawah ini adalah peta atau roadmap latihan di platform TLX Toki:

Modul	Fokus	Konsep kunci
01	Keluaran dasar	<code>cout</code> , teks vs angka vs ekspresi, prioritas operator $((a+b)*2)$
02	Variabel dan string	inisialisasi, <i>re-assignment</i> , urutan tukar nilai, format output
03	Perulangan	<i>for</i> , <i>while</i> , urutan tambah-cetak
04	Percabangan	<code>mod %</code> , <code>if</code> terpisah vs <code>else if</code> , percabangan bersarang maks/min
05	Perulangan dan pola	<code>for</code> rentang, flag, <code>break/continue</code> , nested loop pola grafis
06	Array	array 1D/2D, cari elemen berdasarkan indeks, penerapan dengan looping
07	Fungsi	ekstraksi fungsi, parameter array, referensi, fungsi global

Notes

Peta latihan TLX Toki ini sudah disesuaikan dengan timeline pembelajaran di mata kuliah PPK TRPL, dari bulan Agustus hingga September (sebelum UTS).

11 Glosarium Singkat

Istilah	Arti singkat
cin / cout	pembacaan masukan / pencetakan keluaran
const	deklarasi nilai yang tidak boleh berubah
int	tipe data untuk bilangan bulat
float	tipe data untuk bilangan pecahan dengan presisi tunggal
double	tipe data untuk bilangan pecahan dengan presisi lebih tinggi
char	tipe data untuk menyimpan satu karakter
string	tipe data untuk menyimpan kumpulan karakter atau teks
bool	tipe data logika dengan nilai true atau false
void	tipe yang menunjukkan tidak adanya nilai yang dikembalikan
variable	tempat penyimpanan data yang nilainya dapat berubah
constant	nilai yang tetap selama program berjalan
identifier	nama yang digunakan untuk variabel, fungsi, atau objek
literal	nilai yang ditulis langsung di dalam kode program
operator	simbol yang digunakan untuk melakukan operasi terhadap data
=	operator penugasan nilai ke variabel
+, -, *, /	operator aritmetika dasar
%	operator untuk mendapatkan sisa hasil pembagian
==	operator untuk membandingkan kesamaan dua nilai
!=	operator untuk membandingkan ketidaksamaan dua nilai
<, >	operator perbandingan kurang dari / lebih dari
<=, >=	operator perbandingan kurang atau sama / lebih atau sama
&&	operator logika AND, benar jika kedua kondisi benar
	operator logika OR, benar jika salah satu kondisi benar
!	operator logika NOT untuk membalik nilai logika
if	struktur percabangan berdasarkan suatu kondisi
else	bagian percabangan yang dijalankan jika kondisi if salah
else if	percabangan tambahan untuk memeriksa kondisi lain
switch	struktur percabangan berdasarkan nilai tertentu
case	pilihan kondisi di dalam switch
default	pilihan yang dijalankan jika tidak ada case yang sesuai
for	perulangan dengan jumlah atau kondisi yang ditentukan
while	perulangan selama suatu kondisi bernilai benar
do-while	perulangan yang selalu dijalankan setidaknya satu kali
break	menghentikan perulangan atau switch
continue	melewati iterasi saat ini dan melanjutkan ke iterasi berikutnya
function	blok kode yang dibuat untuk menjalankan tugas tertentu
return	mengembalikan nilai dari sebuah fungsi
parameter	variabel yang digunakan sebagai masukan bagi fungsi
argument	nilai yang diberikan ketika fungsi dipanggil
array	kumpulan data dengan tipe yang sama dan ukuran tertentu
index	posisi elemen dalam array yang dimulai dari indeks 0
string	objek atau tipe data yang digunakan untuk mengolah teks
namespace	ruang lingkup untuk mengelompokkan nama agar tidak bertabrakan
std	namespace standar yang menyediakan berbagai fitur bawaan C++
#include	direktif untuk menyertakan pustaka atau file header

<code>iostream</code>	pustaka untuk operasi masukan dan keluaran standar
<code>using namespace</code>	perintah untuk menggunakan nama dari suatu namespace secara langsung
<code>main()</code>	fungsi utama yang menjadi titik awal eksekusi program
<code>//</code>	penanda komentar satu baris
<code>/* */</code>	penanda komentar yang dapat mencakup beberapa baris
<code>;</code>	tanda yang umumnya menandai akhir sebuah pernyataan C++
<code>{}</code>	tanda kurung kurawal untuk membatasi blok kode
<code>()</code>	tanda kurung yang digunakan pada fungsi dan ekspresi tertentu
<code>[]</code>	tanda kurung siku yang digunakan antara lain untuk array
<code>scope</code>	wilayah program tempat suatu nama dapat digunakan
<code>syntax</code>	aturan penulisan kode yang harus dipatuhi oleh bahasa C++
<code>compiler</code>	program yang menerjemahkan kode C++ menjadi bentuk yang dapat dijalankan
<code>compile</code>	proses menerjemahkan kode sumber menjadi program yang dapat dijalankan
<code>error</code>	kesalahan yang menyebabkan program gagal dikompilasi atau berjalan sesuai harapan
<code>syntax error</code>	kesalahan karena kode tidak mengikuti aturan sintaks C++
<code>runtime error</code>	kesalahan yang terjadi ketika program sedang dijalankan
<code>logic error</code>	kesalahan logika yang membuat hasil program tidak sesuai tujuan
<code>source code</code>	kode program yang ditulis oleh programmer
<code>header</code>	file yang berisi deklarasi yang dapat digunakan dalam program
<code>library</code>	kumpulan kode siap pakai untuk menyediakan fungsi atau fitur tertentu
<code>input</code>	data yang diberikan kepada program sebagai masukan
<code>output</code>	data atau informasi yang dihasilkan oleh program
<code>initialization</code>	proses memberikan nilai awal kepada variabel
<code>declaration</code>	proses memperkenalkan nama dan tipe suatu variabel atau entitas
<code>assignment</code>	proses memberikan atau mengubah nilai suatu variabel

Selamat Belajar!

Semoga sukses di setiap ujian dan tantangan.

Long Live Software Engineering!